[2] Agent-SafetyBench: Evaluating the Safety of LLM Agents.

Students should use the official benchmark implementation and evaluation protocol.

[3] Rimsky et al. Steering Llama 2 via Contrastive Activation Addition.

Introduces Contrastive Activation Addition (CAA), where activation differences from contrasting examples are used to construct behavioral steering directions.

Students are expected to reuse existing open-source implementations.

GPU Requirement

Hardware: 1 × NVIDIA A6000.

Proposed allocation cap: 30 GPU-hours per team for the complete project. This is a planning limit, not a measured runtime estimate.

No model fine-tuning is required. Include activation collection, steering-vector construction, development selection, all final conditions, failed runs, and evaluation. Confirm compatibility and memory use with a complete steering-and-evaluation pilot.

Decision-point steering is optional and must fit within the confirmed allocation; it carries no separate project-specific bonus.

Project 5

Project title: Evolving Graph-Structured Skills for LLM Agents

Project manager: Jiujiu Chen

Email: jchen349@connect.hkust-gz.edu.cn

Project description

LLM-based agents use language models to coordinate reasoning, actions, and feedback when solving tasks. To reuse effective procedures across tasks, agents can maintain reusable skills: explicit instructions or executable programs that can be retrieved and combined. However, existing skills may be incomplete or fail on unfamiliar cases, motivating skill updates that refine procedures using evidence from previous attempts. Feedback-driven approaches demonstrate how agents can improve through external reflective context without changing model parameters. Building on these ideas, this project uses GraphSkillEvo to investigate how graph-structured skills evolve, why they still fail, and which feedback and failure cases should guide subsequent updates.

Codebases and datasets

- **Baseline:** GraphSkillEvo
- **Datasets:** SearchQA (GraphSkillEvo split) and HotpotQA (provided)

Tasks

Part 1: Understand and reproduce the method

Run GraphSkillEvo using one fixed model throughout the comparison. The submitted project proposes an open-source model such as qwen3.5-9B. Initialize skills, execute the bounded evolutionary optimization loop, select the final skill using validation data, and evaluate it on fixed subsets of the provided SearchQA and HotpotQA test sets.

1. Explain the Roles of the Training and Validation Sets

- What information is obtained from training examples, and how does it influence skill updates?
- How are validation results used to rank candidates, select parents and survivors, and choose the final skill?
- Why are separate training and validation sets necessary? What risks arise from repeatedly selecting skills on the same validation set?

2. Report Results on the Provided Datasets

- Report Exact Match (EM), F1, and latency on the two fixed test subsets. Describe answer extraction and normalization, and clearly identify the evaluated subsets.
- Compare with initial-skill and no-skill baselines, analyze their difference in test cases.

3. Trace and Visualize Skill Evolution

- Visualize the skills as a flowchart produced in each iteration, starting from initialization and ending with the selected final skill.
- The flowchart should show the task-type routing, nodes, execution order, and any explicit branches or conditions.
- Discuss which changes appear helpful, ineffective, or harmful, using validation results and recorded examples.

Part 2: Analyze Skill Failures

During method development, construct the complete set of failed development examples on which the baseline selected skill receives EM = 0. Use these failures to

investigate shared problem patterns, distinct groups, and other meaningful commonalities. Do not use held-out test failures to design the optimizer modification.

After all methods and settings are frozen, construct the complete failure inventory for each fixed test subset as a final descriptive analysis. Test failures must not be fed back into skill optimization or prompt selection.

The analysis can include:

- **An error inventory:** example ID, question, reference answer(s), predicted answer, EM/F1, assigned category, and a short diagnostic explanation.
- **A failure taxonomy:** define your categories, report their counts or proportions, and explain how you handle overlapping or uncertain labels.
- **A skill-level diagnosis:** identify missing, ambiguous, ineffective, or unfollowed instructions and explain why the final skill does not resolve the case. Separate evidence-supported explanations from hypotheses.

Part 3: Redesign the skill optimization procedure

Based on the development-set analysis in Part 2, propose one concrete update to GraphSkillEvo's iterative skill optimization procedure. Choose either the optimization inputs or the failure-case selection policy as the primary modification, keeping the other component fixed. Evaluate the frozen modification against the original optimizer on the same held-out test subsets with matched model-call and generation-token budgets. Do not merely rewrite the final skill by hand.

- **Option A — Update the optimization inputs:** Specify what information should be added for the optimizer. Discuss whether successful examples should also be included to help preserve existing capabilities.
- **Option B — Design the failure-case selection policy:** Define an implementable policy for constructing each iteration's optimization batch. Address the source pool, selection criteria, diversity, difficulty, redundancy, persistent failures, and how the pool is refreshed after updates. Explain why your policy should be more informative than the baseline policy you inspected.

Optional Extension: Cross-Dataset Skill Generalization

Choose at most one extension after completing the required comparison and only within the confirmed allocation:

- **Option A:** Transfer Skill Capabilities from SearchQA to HotpotQA. Use the evolved SearchQA skill as the starting point for adaptation to HotpotQA. Identify

reusable capabilities and refine or extend them using HotpotQA training feedback. Compare against both the unadapted SearchQA skill and a skill independently evolved on HotpotQA to assess the benefits of capability transfer.

- **Option B:** Jointly Update Skill Capabilities across SearchQA and HotpotQA. Iteratively update one shared skill using feedback from both datasets. Design how examples and failures are combined, resolve conflicting updates, and evaluate whether joint updates improve complementary capabilities while preserving performance on both tasks.

Part 4: Finish the final report and demonstration

Write a report of up to six pages covering research background, research problem, methodology, experiments, and conclusions. Implementation details may be included in an appendix. Submit the code, fixed data IDs, prompts, skill graphs, evaluation outputs, comparison tables, GPU accounting log, and demonstration materials required by the unified course format.

References

[1] GraphSkillEvo: Evolutionary Optimization of Graph-Structured Agent Skills. arXiv, 2026.

[2] SkillOpt: Executive Strategy for Self-Evolving Agent Skills. Microsoft. 2026

[3] Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 2023.

[4] ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.

GPU hours

Hardware for local inference: 1 × NVIDIA A6000.

Proposed allocation cap: 40 GPU-hours per team for the complete project. This is a planning limit, not a measured runtime estimate. No model-parameter training is required.

For each dataset, use at most 100 training examples for skill updates, 30 validation/development examples for selection and diagnosis, and 50 held-out test examples. Preserve the supplied partition boundaries. Select fixed IDs before inspecting outcomes, and disclose repeated validation use as a limitation.

Use at most three evolution rounds and at most two newly evaluated candidates per round, in addition to any retained current best. Fix execution and generation limits in the pilot. Apply the same limits to the original and modified optimizer, and retain the initial-skill and no-skill baselines.

Cache identical deterministic data access or tool results where valid, but do not count reused model outputs as fresh independent runs. Account for calls used to create, execute, score, and revise skills, not just final test inference.

Project 6

Project title: Prompting and Fine-tuning Language Models for Grid Path Planning

Project manager: Yue Chang

Email: ychang500@connect.hkust-gz.edu.cn

Project description

This project studies whether language models can detect and repair errors in grid-based navigation plans. It builds on the PPNL benchmark introduced in “Can Large Language Models be Good Path Planners?”, where an agent must move from a start location to a goal while avoiding obstacles.

Instead of evaluating only direct path generation, this project introduces a generate–verify–repair pipeline. First, a language model generates or receives a candidate action sequence. A verifier must determine whether the sequence is feasible, identify the first failure location, and classify the failure type, such as moving out of bounds, hitting an obstacle, or stopping before reaching the goal. A repair model must then produce a corrected action sequence using the grid description, the candidate plan, and the verifier feedback.

Training examples will be constructed automatically from valid shortest paths by introducing controlled errors such as action substitution, action deletion, premature stopping, or multiple errors. All generated examples will be checked using the official single-goal executor. The project will compare direct generation, self-verification, and executor-guided repair, and investigate whether models trained on single-error plans can generalize to plans containing multiple errors.

For a feasible undergraduate NLP project, the scope will remain limited to single-goal environments. Students may use prompting or fine-tuning with small encoder-decoder models such as Flan-T5-small, Flan-T5-base, or BART-base. The main research